



UNIVERSITY OF NAIROBI

DATA STRUCTURES

PREPARED BY ISAAK KAMAU

[linkedin.com/in/isaak-mwangi/](https://www.linkedin.com/in/isaak-mwangi/) | https://x.com/aidev_isaak

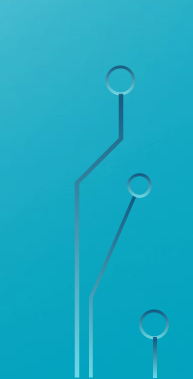
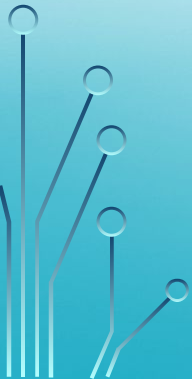


• In this session, we are shall be covering:

• Python Data Structures

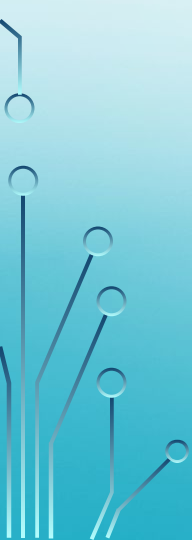
• Built-in data/ Standard Data types(lists, sets, tuples and dictionary)

• Python strings: Replace, Join, Split, Reverse.



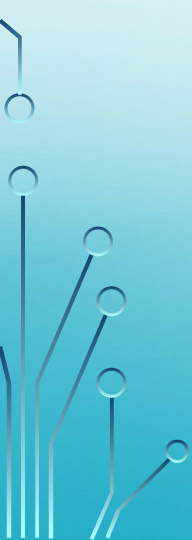


DATA STRUCTURES

- Data Structures
 - Welcome to this lesson on data structures! In the previous lesson, you learned about many of the basic building blocks, including data types, for programming in Python. In this lesson, you will learn to make use of new data structures, which group and order data in different ways, to help you solve problems.
 - You'll learn about:
 - Types of Data Structures: Lists, Tuples, Sets, Dictionaries, Compound Data Structures
 - Operators: Membership, Identity
 - Built-In Functions and Methods
- 

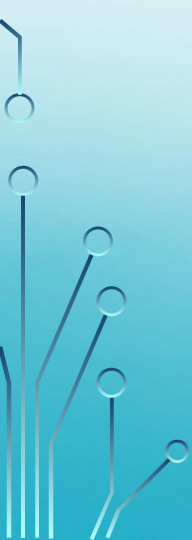


WHAT ARE DATA STRUCTURES?

- **Data structures** are containers or collections of data that organize and group data types together in different ways. You can think of data structures as file folders that have organized files of data inside them.
- 
- 



LIST

- Lists are used to store multiple items in a single variable.
 - Lists are one of 4 built-in data types in Python used to store collections of data, the other 3 are Tuples, Set, and Dictionary, all with different qualities and usage.
 - Lists are created using square brackets:
- 
- 
- 

EXAMPLE

- `list_of_random_things = [1, 3.4, 'a string', True]`
- This is a list of 4 elements. All ordered containers (like lists) are indexed in Python using a starting index of 0. Therefore, to pull the first value from the above list, we can write
- `>>> list_of_random_things[0] 1`

- you can retrieve the last element by reducing the index by 1.
Therefore, you can do the following:

- ```
>>> list_of_random_things[len(list_of_random_things) - 1] True
```

- Alternatively, you can index from the end of a list by using negative values, where -1 is the last element, -2 is the second to last element and so on.

- ```
>>> list_of_random_things[-1] True
```

- ```
>>> list_of_random_things[-2] a string
```

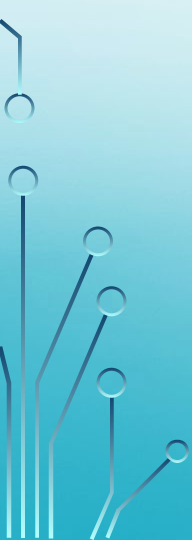


# SLICE AND DICE WITH LISTS

- We can pull more than one value from a list at a time by using **slicing**. When using slicing, it is important to remember that the "lower" index is "Inclusive" and the "Upper" index is "Exclusive"
- ```
>>> list_of_random_things = [1, 3.4, 'a string', True]  
>>> list_of_random_things[1:2] [3.4]
```
- will only return **3.4** in a list. Notice this is still different than just indexing a single element, because you get a list back with this indexing. The colon tells us to go from the starting value on the left of the colon up to, but not including, the element on the right.



MUTABILITY AND ORDER

- **Mutability** refers to whether or not we can change an object once it has been created. If an object can be changed, it is called **mutable**. However, if an object cannot be changed after it has been created, then the object is considered **immutable**.
- 
- 

EXAMPLES - LISTS ARE MUTABLE

- `>>> my_lst = [1, 2, 3, 4, 5]`
- `>>> my_lst[0] = 'one'`
- `>>> print(my_lst) ['one', 2, 3, 4, 5]`
- With the list above, you are able to replace '1' with 'one'. This is because lists are **mutable**.

METHODS

- Join Method

```
name = "-".join(["García", "O'Kelly", "Davis"])  
print(name)
```

- Output:

- García-O'Kelly-Davis

A decorative graphic of circuit lines and nodes is positioned on the left side of the slide, extending from the top to the bottom. The lines are thin and light blue, with small circles at the nodes.

- Append Method

- `letters = ['a', 'b', 'c', 'd']` `letters.append('z')`
`print(letters)`

- Output:

- `['a', 'b', 'c', 'd', 'z']`

TUPLES

- A tuple is another useful container. It's a data type for immutable ordered sequences of elements. They are often used to store related pieces of information. Consider this example involving latitude and longitude:
- `location = (13.4125, 103.866667)`
- `print("Latitude:", location[0])`
- `print("Longitude:", location[1])`

- Tuples can also be used to assign multiple variables in a compact way.
- The parentheses are optional when defining tuples, and programmers frequently omit them if parentheses don't clarify the code.
- `length, width, height = 52, 40, 100`
- `print("The dimensions are {} x {} x {}".format(length, width, height))`

SETS

- A **set** is a data type for mutable unordered collections of unique elements. One application of a set is to quickly remove duplicates from a list.
- `numbers = [1, 2, 6, 3, 1, 1, 6]`
- `unique_nums = set(numbers)`
- `print(unique_nums)`
- Output:
- `{1, 2, 3, 6}`



• Example 2:

• `myset = {"apple", "banana", "cherry"}`



SETS METHODS

- `fruit = {"apple", "banana", "orange", "grapefruit"}` #
define a set
- `print("watermelon" in fruit)` # *check for element*
- `fruit.add("watermelon")` # *add an element*
- `print(fruit) print(fruit.pop())` # *remove a random element*
- `print(fruit)`

DICTIONARIES

- A **dictionary** is a mutable data type that stores mappings of unique **keys** to **values**. Here's a dictionary that stores elements and their atomic numbers.
- `elements = {"hydrogen": 1, "helium": 2, "carbon": 6}`
- we could print out the atomic number mapped to helium like this:
- `print(elements["helium"])`

- We can also insert a new element into a dictionary as in this example:

- `elements["lithium"] = 3`

- `print(elements)`

- Output:

- `{'hydrogen': 1, 'carbon': 6, 'helium': 2, 'lithium': 3}`

DICTIONARY METHODS

- `print("carbon" in elements)`

- Output:

True

- `print(elements.get("dilithium"))`

- Output:

- None

- "carbon" is in the dictionary, so True is printed. "dilithium" isn't in

CONCLUSION:

Data Structure	Ordered	Mutable	Constructor	Example
List	Yes	Yes	[] or list()	[5.7, 4, 'yes', 5.7]
Tuple	Yes	No	() or tuple()	(5.7, 4, 'yes', 5.7)
Set	No	Yes	{ }* or set()	{5.7, 4, 'yes'}
Dictionary	No	No**	{ } or dict()	{'Jun': 75, 'Jul': 89}